

Computación Paralela- Laboratorio 4 - CUDA

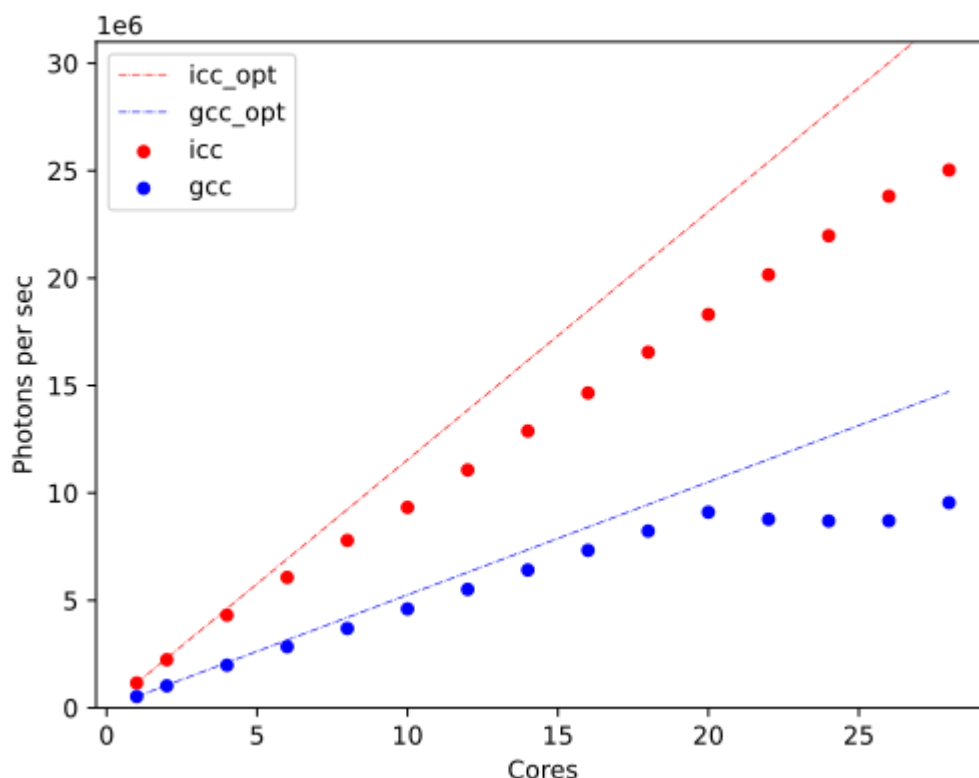
Computación paralela 2021-FAMAF

Cremona Javier
Castillo Marcelo

Resultados del Laboratorio 3 en JupiterAce

Para el laboratorio 3, el mejor resultado obtenido fue de 25 M photons per second (pps), usando el compilador icc, para 32768000 fotones simulados y 28 cores.

Average photons per sec (25 samples, 32768000 photons, 16 lanes)



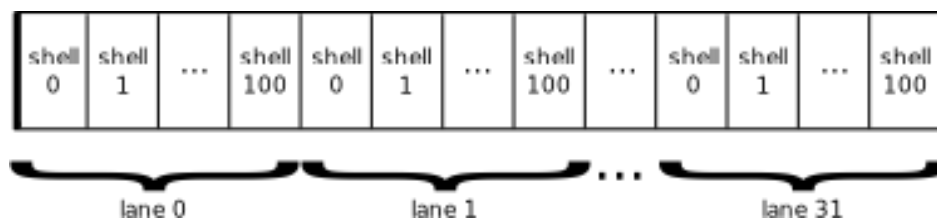
Laboratorio 4

El objetivo del presente trabajo es portar el problema de tiny_mc a CUDA para sacar provecho de la GPU. Los resultados mostrados se obtuvieron en jupiterace con una placa RTX 2080 Ti, ya que se obtuvo mejor performance que con la RTX 3070 de zx81.

Se propusieron 3 distintas soluciones. Inicialmente, se decidió utilizar un RNG por hilo y procesar múltiples fotones por hilo. De esta manera se utiliza el mismo RNG para varios fotones y también mejora el balanceo de carga. Además, se mantienen arreglos heat y heat2 en shared memory para ir almacenando resultados parciales. Estos resultados se vuelcan en los arreglos globales una vez que finaliza de

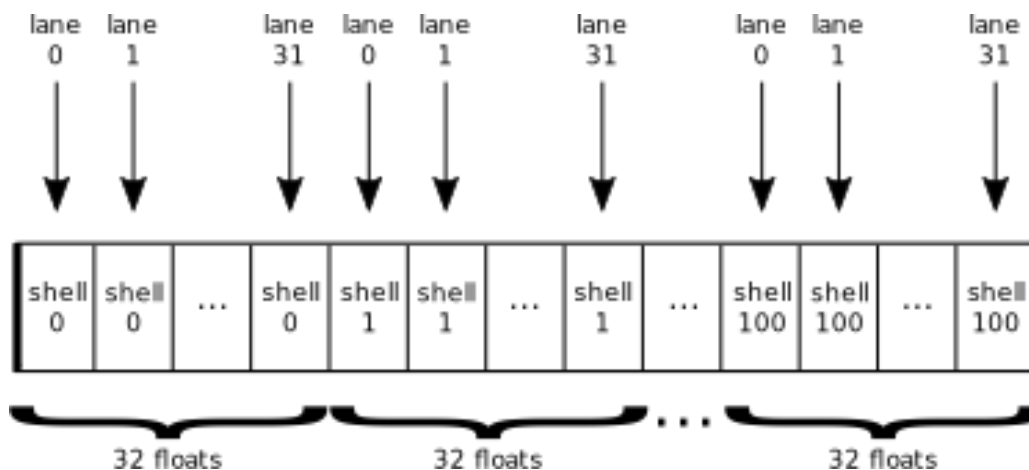
ejecutar un bloque. En particular, tanto los arreglos globales como los RNGs son inicializados en el host, antes de lanzar el kernel.

Como múltiples hilos acceden a los arreglos en shared memory en simultáneo, se utiliza atomicAdd. Esto introduce un retardo, ya que se bloquea a los hilos si hay otro escribiendo. Para mitigar este problema, se propone una segunda solución que utiliza más shared memory. En particular, se replican 32 veces los arreglos heat y heat2 en shared memory. De esta forma, los hilos chocan menos veces al intentar escribir en estos arreglos. Para este caso, se decidió darle un arreglo por cada lane de un warp, como puede verse en la siguiente figura:



Luego, todas las lane 0 de los warps del bloque escriben en los primeros 101 elementos del arreglo compartido, las lane 1 de los warps del bloque escriben en los segundos 101 elementos y así sucesivamente. En total, ambos arreglos ocupan aproximadamente 25 KB de shared memory. El límite en la RTX 2080 ti es de 48 KB de shared memory.

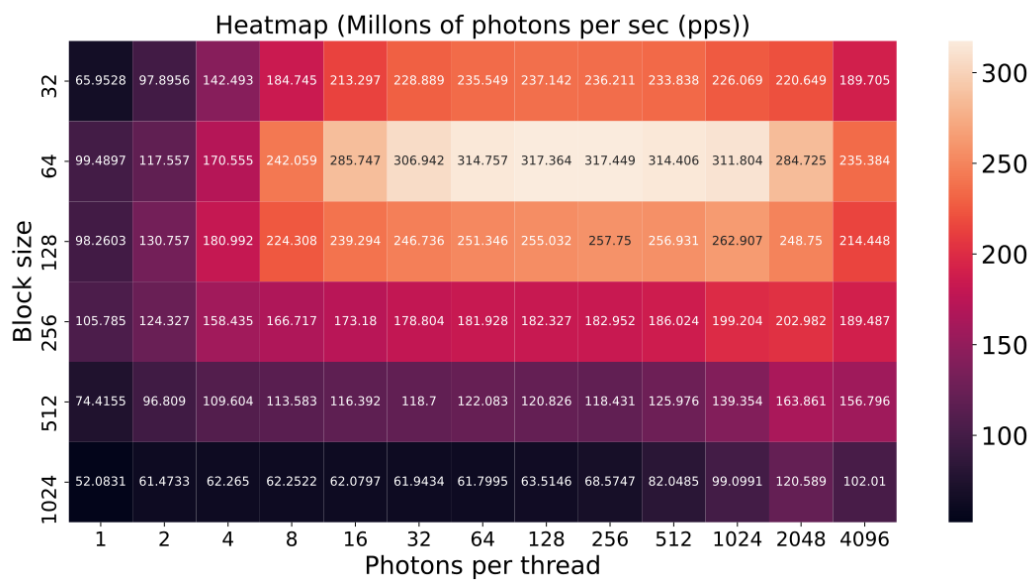
La tercera solución utiliza los mismos arreglos pero el almacenamiento de los datos es distinto. Ahora los lanes escriben de la siguiente manera:



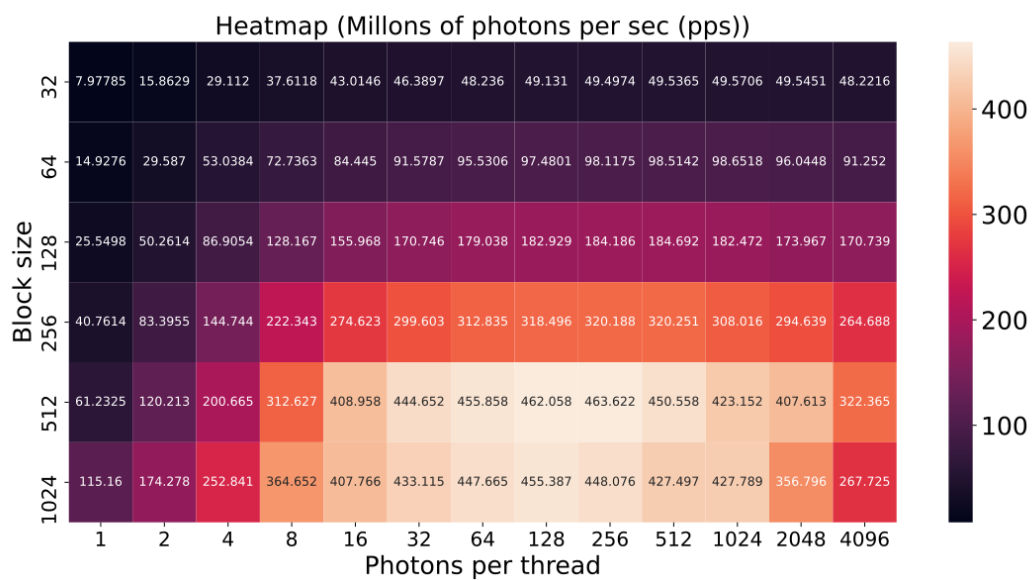
Resultados

Se muestran resultados para $(3.2768 * 1e8)$ fotones, en forma de heat map, en el que varían el tamaño de bloques y la cantidad de fotones simulados por hilo.

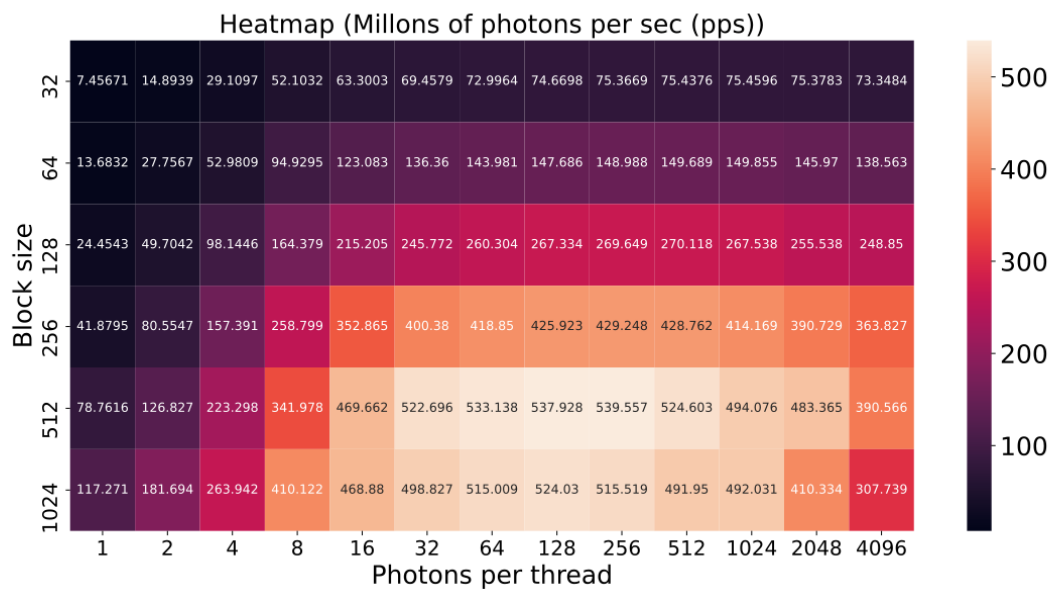
Solución 1:



Solución 2:

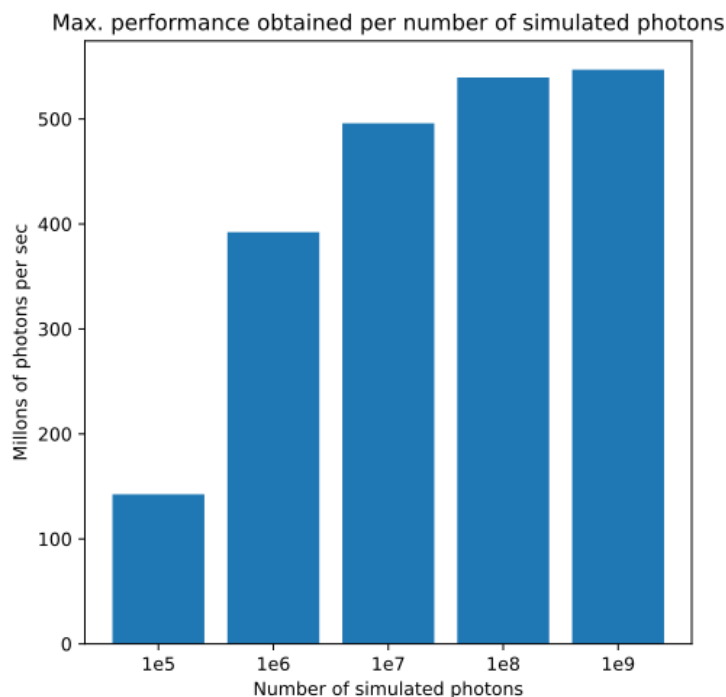


Solución 3:



Puede observarse que las soluciones 2 y 3 mejoran con bloques de mayor tamaño que la solución 1. Esto es probablemente porque emplean mayor cantidad de shared memory, y no se le puede tirar muchos bloques de tamaño chico a un Streaming Multiprocessor porque no entrarían. Conviene entonces, hacer más trabajo por bloque.

Para la solución 3 variamos el tamaño del problema (cantidad de fotones simulados). En cada caso se reporta el óptimo (los cuales se consiguen con distintos tamaños de bloque y distinta cantidad de fotones por hilo).



Vemos que para $1e5$ fotones simulados la performance cae bastante. Probablemente no le estamos dando demasiado trabajo para realizar a la GPU, y no la estamos manteniendo lo suficientemente ocupada como para rendir de mejor manera.

En cuanto al Roofline, reportó una performance de 3 TFLOP/s, contra un techo de 11 TFLOP/s (para la solución 3 con $3.2 * 1e8$ fotones simulados).

En conclusión, se propusieron tres soluciones para resolver el problema de tiny_mc, cada una presentó una mejora respecto a la anterior. La solución 3 presentó mejores resultados. Al parecer, esa forma de repartir la memoria compartida genera menos contención que la de la solución 2. Se obtuvo un pico de 550 M de pps. También vimos que la performance decae para menor cantidad de fotones simulados, probablemente porque la GPU no está lo suficientemente ocupada como para rendir de manera óptima. Una posible mejora es tratar de aumentar la cantidad de memoria shared a usar. Probamos con arreglos de $(101 * 37)$ pero no obtuvimos mejoras, y con arreglos de $(101 * 64)$ nos topamos con el máximo posible a usar de memoria shared.